WEEK 5 ASSIGNMENT

Solved answers in simple language | Aptitude, LeetCode and System Design

1. Aptitude – 10 Questions

1) Ratio of ages

Let the present ages of A and B be 4x and 5x. After 6 years:
(4x + 6) / (5x + 6) = 5 / 6
6(4x + 6) = 5(5x + 6)
24x + 36 = 25x + 30, so x = 6.
A's age = 4 × 6 = 24 years. B's age = 5 × 6 = 30 years.

Final answer: A = 24 years; B = 30 years.

2) Difference between compound interest and simple interest

Principal = ■10,000, rate = 10% per year, time = 2 years.
Simple Interest = (P × R × T) / 100 = (10,000 × 10 × 2) / 100 = ■2,000.
Compound Amount = 10,000 × (1 + 10/100)² = ■12,100.
Compound Interest = 12,100 – 10,000 = ■2,100.
Difference = 2,100 – 2,000 = ■100.

Final answer: ■100.

3) Probability that two dice give a sum of 8

When two dice are rolled, total possible outcomes = 6 × 6 = 36.
The outcomes with sum 8 are (2,6), (3,5), (4,4), (5,3), and (6,2). There are 5 favourable outcomes.
Probability = favourable outcomes / total outcomes = 5/36.

Final answer: 5/36.

4) Two pipes: one fills and one empties the tank

Pipe A fills 1/10 of the tank in one hour. Pipe B empties 1/15 of the tank in one hour.
Net work in one hour = 1/10 – 1/15 = (3 – 2)/30 = 1/30 of the tank.
Therefore, the time to fill one full tank = 30 hours.

Final answer: 30 hours.

5) Riya's relation to the man

The man's mother is described as the only daughter of Riya's mother. If Riya is that only daughter, then Riya is the man's mother.

Final answer: Riya is the man's mother.

6) Find the other number using LCM and HCF

For two positive integers: First number × Second number = LCM × HCF.
12 × Second number = 48 × 4 = 192.
Second number = 192 / 12 = 16.

Final answer: 16.

7) Distance to the office

At 4 km/h the person is 10 minutes late. At 5 km/h the person is 5 minutes early. So, the difference between the two travel times is 15 minutes = 1/4 hour.

Let distance be d km. Then d/4 – d/5 = 1/4.
d(1/20) = 1/4, so d = 5 km.

Final answer: 5 km.

8) Percentage who passed both subjects

60% passed English and 70% passed Maths. 10% failed both, so 90% passed at least one subject.
Using: Passed English + Passed Maths – Passed both = Passed at least one.
60 + 70 – Both = 90, so Both = 40%.

Final answer: 40%.

9) Four-digit numbers divisible by 5 without repeated digits

A number divisible by 5 must end in 0 or 5.
Case 1 – Last digit is 0: The first digit has 9 choices (1–9). The middle two positions have 8 and 7 choices. Count = 9 × 8 × 7 = 504.
Case 2 – Last digit is 5: The first digit cannot be 0 or 5, so it has 8 choices. The middle two positions have 8 and 7 choices. Count = 8 × 8 × 7 = 448.
Total = 504 + 448 = 952.

Final answer: 952 numbers.

10) Probability of getting at least one six in four throws

It is easier to calculate the opposite event: getting no six in all four throws.
Probability of no six in one throw = 5/6.
Probability of no six in four independent throws = (5/6)■ = 625/1296.
Probability of at least one six = 1 – 625/1296 = 671/1296.

Final answer: 671/1296 (approximately 0.518).

2. LeetCode – Mandatory Problems

The following solutions use Python. Each code block can be submitted separately on LeetCode.

56. Merge Intervals (Medium)

Sort intervals by their starting value. If the next interval overlaps the last merged interval, combine them. Otherwise, add it as a new interval.

```
class Solution:
    def merge(self, intervals):
        intervals.sort(key=lambda x: x[0])
        merged = []

        for start, end in intervals:
            if not merged or start > merged[-1][1]:
                merged.append([start, end])
            else:
                merged[-1][1] = max(merged[-1][1], end)

        return merged
```

Complexity: Time: O(n log n) for sorting. Space: O(n) for the output.

49. Group Anagrams (Medium)

Anagrams contain the same letters with the same frequencies. Sort the letters of each word and use that sorted word as a dictionary key.

```
from collections import defaultdict

class Solution:
    def groupAnagrams(self, strs):
        groups = defaultdict(list)

        for word in strs:
            key = ''.join(sorted(word))
            groups[key].append(word)

        return list(groups.values())
```

Complexity: If there are n words of average length k: Time O(n × k log k). Space O(n × k) for stored words.

102. Binary Tree Level Order Traversal (Medium)

Use a queue for breadth-first search. Process all nodes currently in the queue as one level, then add their children for the next level.

```
from collections import deque

class Solution:
    def levelOrder(self, root):
        if not root:
            return []

        result = []
        queue = deque([root])

        while queue:
            level = []

            for _ in range(len(queue)):
                node = queue.popleft()
                level.append(node.val)

                if node.left:
                    queue.append(node.left)
                if node.right:
                    queue.append(node.right)

            result.append(level)

        return result
```

Complexity: Time: O(n), because each node is visited once. Space: O(w), where w is the maximum number of nodes at one level.

23. Merge k Sorted Lists (Hard) – Optional

Use a min-heap. The heap always gives the smallest current node among the k lists. Remove that node, add it to the answer, and insert its next node if it has one.

```
import heapq

class Solution:
    def mergeKLists(self, lists):
        heap = []

        for i, node in enumerate(lists):
            if node:
                heapq.heappush(heap, (node.val, i, node))

        dummy = ListNode(0)
        tail = dummy

        while heap:
            value, i, node = heapq.heappop(heap)
            tail.next = node
            tail = tail.next

            if node.next:
                heapq.heappush(
                    heap, (node.next.val, i, node.next)
                )

        tail.next = None
        return dummy.next
```

Complexity: If there are N total nodes and k lists, time is O(N log k). The heap uses O(k) extra space.

Note: The Hard problem is optional, as stated in the assignment.

3. System Design – Design an API Rate Limiter

1) What is an API rate limiter?

An API rate limiter controls how many requests a user or application can send in a given time. For example, an API may allow 100 requests per minute for one user. If the user sends more requests, the system can return HTTP 429 (Too Many Requests). This protects the API from overload and misuse.

2) Compare the three approaches

Approach	How it works	Advantages	Limitations / best use
Token Bucket	Tokens are added to a bucket at a fixed rate. Each request uses a token. If no token is available, the request is rejected or delayed.	Allows short bursts while controlling the average rate.	Good default for APIs where small bursts are acceptable.
Leaky Bucket	Requests enter a queue and are processed at a steady rate. If the queue is full, new requests are rejected.	Smooths traffic and gives a stable output rate.	Can add waiting time; useful when downstream systems need steady traffic.
Sliding Window	Counts requests in a moving time window, such as the last 60 seconds.	More accurate than simple fixed windows.	Can use more memory or processing; useful for precise limits.

3) Where should the rate limiter sit?

A common design places the rate limiter at the API Gateway, before requests reach backend services. The gateway checks the request, identifies the user or API key, and asks the rate-limiter logic whether the request is allowed. If allowed, the request is forwarded to the service. If the limit is exceeded, the gateway returns HTTP 429. This central location applies limits consistently to many services. A service can also apply an additional limiter for sensitive operations.

4) How are counters stored in Redis?

Redis is a fast shared in-memory data store. Each user and API limit can have a separate key. For example, a sliding-window counter could use a key such as `rate:user123:orders`. The system stores request timestamps or time-bucket counts and removes expired data. A key expiry (TTL) automatically removes old counters. For a token bucket, Redis can store the available token count and the time of the last refill.

Important: updating a counter and checking the limit should happen as one atomic operation. Redis Lua scripts or an atomic Redis command pattern can prevent two requests from racing and both passing the limit.

5) How does it stay consistent across multiple server instances?

All gateway or application instances use the same shared Redis store instead of keeping separate local counters. Therefore, requests handled by different servers update the same user's rate-limit data. For example, if server A processes 40 requests and server B processes 30 requests, Redis keeps the shared total of 70.

To improve consistency and reliability:
• Use atomic Redis operations or Lua scripts for checking and updating limits.
• Use the same key format and limit configuration on every instance.
• Set TTL values so old counters are cleaned up.
• Use Redis replication or clustering for availability and scale, while understanding that failover and replication settings can affect the strictness of limits.
• Decide what to do if Redis is unavailable: fail open (allow requests) or fail closed (reject requests). The choice depends on whether availability or strict protection is more important.

6) Simple request flow

Step	Action
1	Client sends an API request to the API Gateway.
2	Gateway identifies the user/API key and selects the correct limit.
3	Rate limiter checks and updates the user's data in Redis atomically.

Step	Action
4	If within the limit, the gateway forwards the request to the backend service.
5	If the limit is exceeded, the gateway returns HTTP 429 and may include a Retry-After header.

Conclusion: An API rate limiter protects services by controlling request traffic. A practical design puts the limiter at the API Gateway and uses Redis as shared storage. Token Bucket is flexible for bursts, Leaky Bucket smooths traffic, and Sliding Window provides accurate time-based counting.